DEPARTMENT OF CSE, CS&IT, AI&DS
COURSE CODE: 24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L
FULL STACK APPLICATION DEVELOPMENT

Date of the Session: __/__/____ **Time of The Session: _____to_____**

#SKILL - 9 ➔ Global Exception Handling using @ControllerAdvice

Prerequisites:

- **Understanding of REST API exception flow**
- **Concept of custom exceptions**

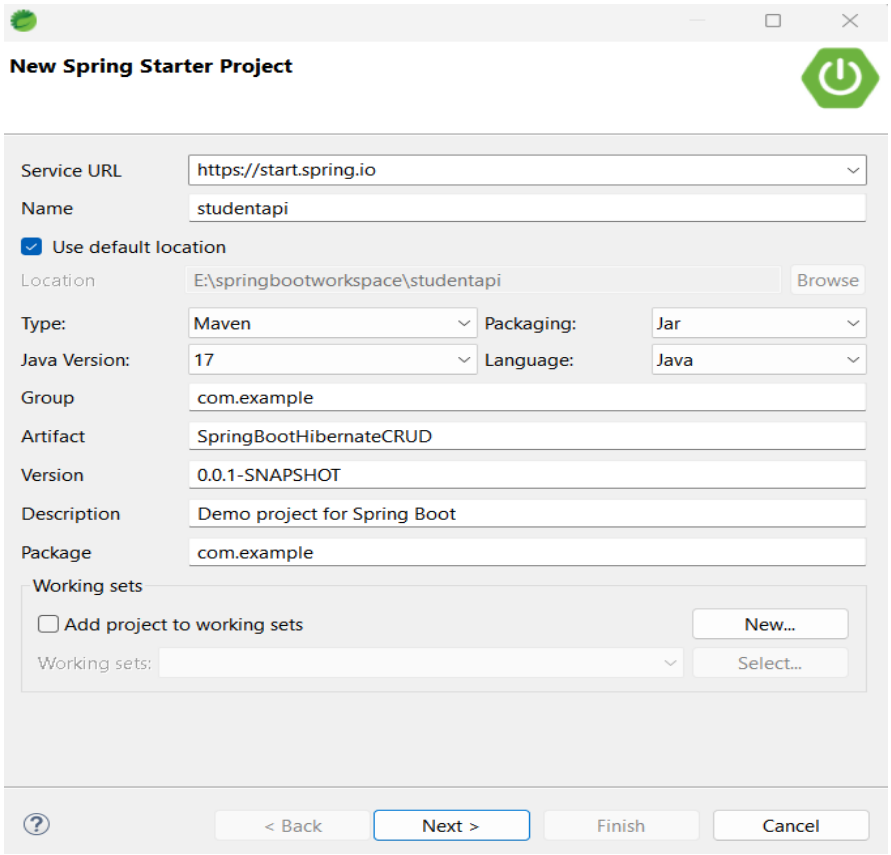
A university application provides **student details** through REST APIs. When users enter an **invalid student ID**, the system shows technical error messages that are difficult for users to understand. To improve usability, the application should return **clear, readable, and user-friendly error responses** through **centralized exception handling**.

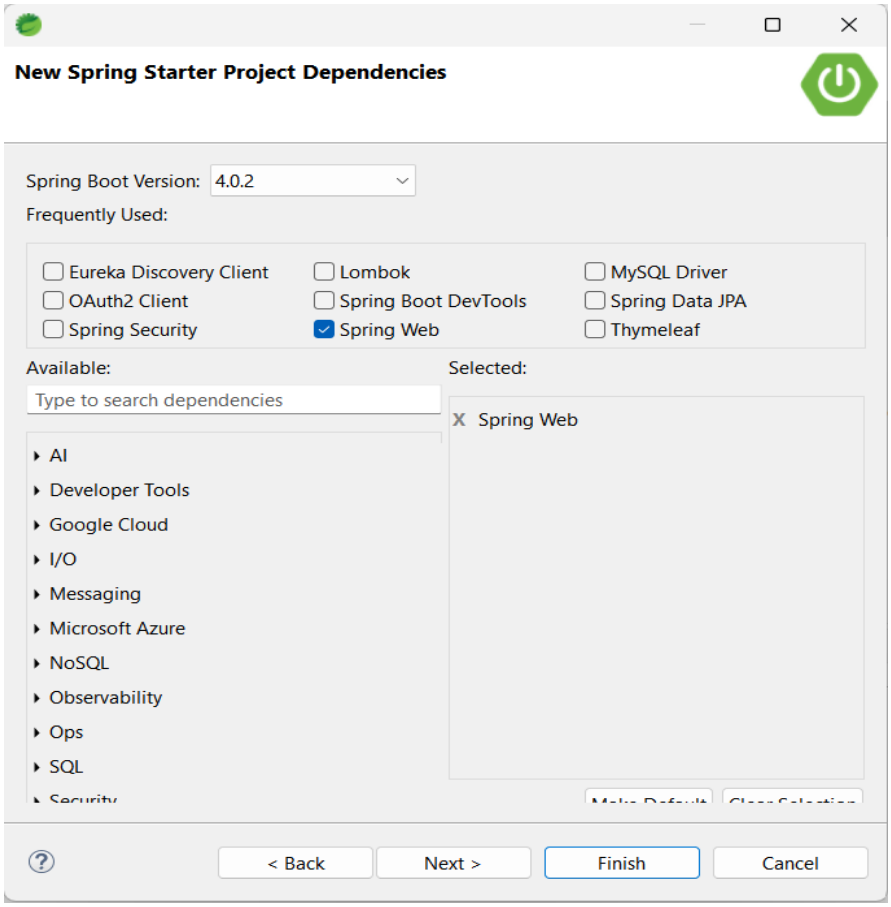
Your task is to implement **Global Exception Handling** using **@ControllerAdvice** and create custom exceptions for invalid student inputs.

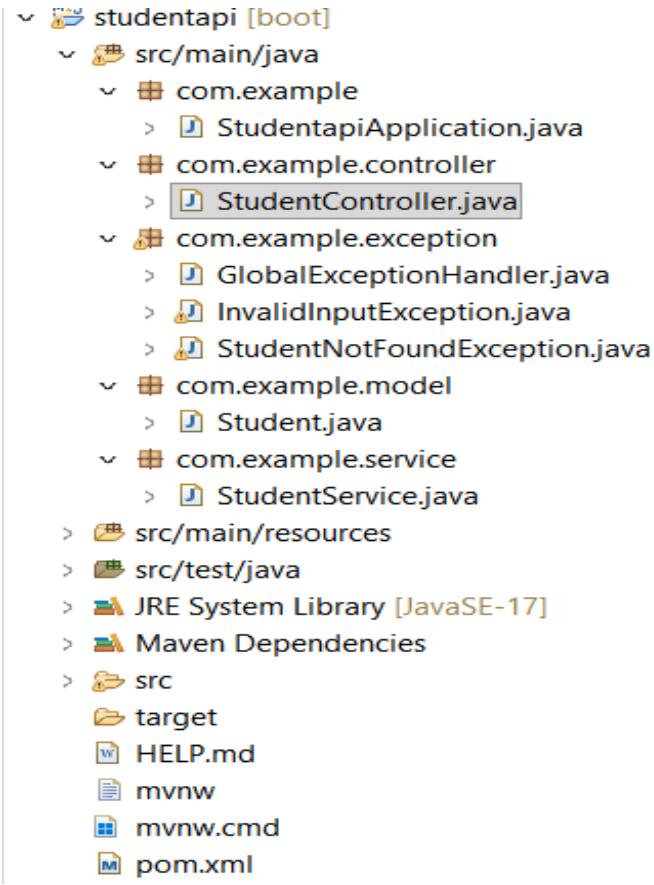
Tasks:

1. Create a custom exception class named **StudentNotFoundException**.
2. Create a REST endpoint **/student/{id}** that retrieves student information and **throws the custom exception** when the given student ID is invalid.
3. Create a **GlobalExceptionHandler** class using **@ControllerAdvice** to handle exceptions across the entire application.
4. Add a method with **@ExceptionHandler(StudentNotFoundException.class)** to return a **meaningful error response** along with an appropriate HTTP status code.
5. **Add a second custom exception**, such as **InvalidInputException**, and handle it inside the same @ControllerAdvice class.
6. **Return a structured JSON response** (fields like timestamp, message, statusCode) from the exception handler to make the error readable and consistent.
7. Test the API with **valid and invalid student IDs** and observe the difference in responses.
8. Test an endpoint with **invalid input format** (e.g., sending text instead of a number) to trigger the second custom exception **using Postman**.
9. **Push the Spring Boot project to GitHub.**

Solution:


New Spring Starter Project
Service URL
https://start.spring.io
Name
studentapi
Use default location
Location
E:\springbootworkspace\studentapi
Browse
Type:
Maven
Packaging:
Jar
Java Version:
17
Language:
Java
Group
com.example
Artifact
SpringBootHibernateCRUD
Version
0.0.1-SNAPSHOT
Description
Demo project for Spring Boot
Package
com.example
Working sets
Add project to working sets
New...
Working sets:
Select...
< Back
Next >
Finish
Cancel


New Spring Starter Project Dependencies
Spring Boot Version:
4.0.2
Frequently Used:
Eureka Discovery Client
Lombok
MySQL Driver
OAuth2 Client
Spring Boot DevTools
Spring Data JPA
Spring Security
Spring Web
Thymeleaf
Available:
Selected:
Type to search dependencies
X Spring Web
AI
Developer Tools
Google Cloud
I/O
Messaging
Microsoft Azure
NoSQL
Observability
Ops
SQL
< Back
Next >
Finish
Cancel



StudentController.java

```
package com.example.controller;


import org.springframework.web.bind.annotation.*;


import com.example.exception.InvalidInputException;
import com.example.model.Student;
import com.example.service.StudentService;


@RestController
@RequestMapping("/student")
public class StudentController {


    private final StudentService service;
```

```
    public StudentController(StudentService service) {
        this.service = service;
    }

    @GetMapping("/{id}")
    public Student getStudent(@PathVariable String id) {

        if (!id.matches("\\d+")) {
            throw new InvalidInputException("Student ID must be numeric");
        }

        int studentId = Integer.parseInt(id);
        return service.getStudentById(studentId);
    }
}
```

Student.java

```
package com.example.model;

public class Student {
    private int id;
    private String name;
    private String department;

    public Student(int id, String name, String department) {
        this.id = id;
        this.name = name;
        this.department = department;
    }

    public int getId() { return id; }
```

```
    public String getName() { return name; }
    public String getDepartment() { return department; }
}
```

StudentNotFoundException.java

```
package com.example.exception;

public class StudentNotFoundException extends RuntimeException {
    public StudentNotFoundException(String message) {
        super(message);
    }
}
```

InvalidInputException.java

```
package com.example.exception;

public class InvalidInputException extends RuntimeException {
    public InvalidInputException(String message) {
        super(message);
    }
}
```

StudentService.java

```
package com.example.service;

import java.util.*;

import org.springframework.stereotype.Service;

import com.example.exception.StudentNotFoundException;
```

```
import com.example.model.Student;

@Service
public class StudentService {

    private static final Map<Integer, Student> STUDENTS = new HashMap<>();

    static {
        STUDENTS.put(101, new Student(101, "Anita", "CSE"));
        STUDENTS.put(102, new Student(102, "Rahul", "ECE"));
        STUDENTS.put(103, new Student(103, "Sita", "IT"));
    }

    public Student getStudentById(int id) {
        if (!STUDENTS.containsKey(id)) {
            throw new StudentNotFoundException("Student not found with ID: " + id);
        }
        return STUDENTS.get(id);
    }
}
```

GlobalExceptionHandler.java

```
package com.example.exception;

import java.time.LocalDateTime;
import java.util.*;

import org.springframework.http.*;
import org.springframework.web.bind.annotation.*;
```

```
@ControllerAdvice
public class GlobalExceptionHandler {

  @ExceptionHandler(StudentNotFoundException.class)
  public ResponseEntity<Map<String, Object>> handleNotFound(StudentNotFoundException
ex) {

    Map<String, Object> error = new HashMap<>();
    error.put("timestamp", LocalDateTime.now());
    error.put("statusCode", HttpStatus.NOT_FOUND.value());
    error.put("message", ex.getMessage());

    return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
  }

  @ExceptionHandler(InvalidInputException.class)
  public ResponseEntity<Map<String, Object>> handleInvalid(InvalidInputException ex) {

    Map<String, Object> error = new HashMap<>();
    error.put("timestamp", LocalDateTime.now());
    error.put("statusCode", HttpStatus.BAD_REQUEST.value());
    error.put("message", ex.getMessage());

    return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
  }
}
```

Output:

localhost:8080/student/101

Pretty-print

{"id":101,"name":"Anita","department":"CSE"}

localhost:8080/student/102

Pretty-print

{"id":102,"name":"Rahul","department":"ECE"}

localhost:8080/student/103

Pretty-print

{"id":103,"name":"Sita","department":"IT"}